Structured Programming Lab

Bachelor Program Business Informatics

Department of Computer Science

Calculations

Integer Arithmetic

IntegerArithmeticDemo Code

```java
public class IntegerArithmeticDemo {

    public static void main(String[] args) {
        System.out.printf("%d + %d = %d\n", 19, 4, 19 + 4);
        System.out.printf("%d - %d = %d\n", 19, 4, 19 - 4);
        System.out.printf("%d * %d = %d\n", 19, 4, 19 * 4);
        System.out.printf("%d / %d = %d\n", 19, 4, 19 / 4);
        System.out.printf("%d %% %d = %d\n", 19, 4, 19 % 4);
        //System.out.printf("%d / %d = %d\n", 19, 0, 19 / 0);

        System.out.printf("%d\n", Integer.MAX_VALUE);
        System.out.printf("%d\n", Integer.MIN_VALUE);
        System.out.printf("%d\n", Integer.MAX_VALUE + 1);
        System.out.printf("%d\n", 19 + 2 * 3);
        System.out.printf("%d\n", (19 + 2) * 3);
    }
}
```

IntegerArithmeticDemo Output

```
19 + 4 = 23
19 - 4 = 15
19 * 4 = 76
19 / 4 = 4
19 % 4 = 3
2147483647
-2147483648
-2147483648
25
63
```

IntegerArithmetic Summary

- integer division returns integer result
- modulo (%) is the remainder after division
- division by zero produces an error (an exception)

```
Exception in thread "main" java.lang.ArithmeticException: / by
 at Expressions.IntegerArithmeticDemo.main(IntegerArithmeticDe
```

- minimum and maximum `int` values (`Integer.MIN_VALUE` and `Integer.MAX_VALUE`)
- over/underflow error (see Two's Complement)
- multiplication and division are evaluated before addition/subtraction in longer expressions
- Use parenthesis to control evaluation sequence

Double Arithmetic

DoubleArithmeticDemo Code

```java
public class DoubleArithmeticDemo {

    public static void main(String[] args) {
        System.out.printf("%f + %f = %f\n", 19., 4., 19. + 4.);
        System.out.printf("%f - %f = %f\n", 19., 4., 19. - 4.);
        System.out.printf("%f * %f = %f\n", 19., 4., 19. * 4.);
        System.out.printf("%f / %f = %f\n", 19D, 4D, 19D / 4D);

        System.out.printf("%g\n", Double.MAX_VALUE);
        System.out.printf("%g\n", Double.MIN_VALUE);

        System.out.printf("%g\n", Double.MAX_VALUE - 1);

        System.out.printf("%f / %f = %f\n", 19., 0., 19. / 0.);
        System.out.printf("%f / %f = %f\n", 0., 0., 0. / 0.);

        System.out.printf("%f\n", Double.POSITIVE_INFINITY * 42);
        System.out.printf("%f\n", Double.NaN * 42);
        System.out.printf("%f\n", Double.POSITIVE_INFINITY * Double.NaN);
    }
}
```

DoubleArithmeticDemo Output

```
19.000000 + 4.000000 = 23.000000
19.000000 - 4.000000 = 15.000000
19.000000 * 4.000000 = 76.000000
19.000000 / 4.000000 = 4.750000
1.79769e+308
4.90000e-324
1.79769e+308
19.000000 / 0.000000 = Infinity
0.000000 / 0.000000 = NaN
Infinity
NaN
NaN
```

DoubleArithmetic Summary

- use a dot (`.`) or a `D` to generate a double value
- use %g to format large `double` values
- adding/subtracting (rather) small values to/from very large values does not necessarily give new values
- Division by zero gives `Infinity`
- most calculations involving `Infinity` give `Infinity` (except when combined with `NaN`)
- `0./0.` gives "Not a number" (`NaN`)
- any calculation involving `NaN` gives `NaN`
- numbers are internally represented by **mantissa** and **exponent**
- mantissa saves (the first) 12-14 digits
- exponent saves "how many places to shift the decimal separator"
- double values have a certain **precision**, calculation results are generally not exact

Mixed Arithmetic Expressions

MixedArithmeticDemo Code

```
1  public class MixedArithmeticDemo {
2
3      public static void main(String[] args) {
4          System.out.printf("%f%n", 1. + 3 /4 );
5          System.out.printf("%f%n", 1  + 3./4 );
6          System.out.printf("%f%n", 1  + 3 /4.);
7          System.out.printf("%f%n", 1  + (double) 3 /4);
8          System.out.printf("%d%n", (int) (1  +  3. /4));
9      }
10 }
```

MixedArithmeticDemo Console

```
1.000000
1.750000
1.750000
```

Mixed Expressions Summary

- if one of the operands is `double`, the other is converted to `double` **before** the operation
- convert to a different type with the *type cast operator**
  - `(int)` convert to `int` (information loss!)
  - `(double)` convert to `double`
- type casting should be the exception, *not* the rule

Types

Primitive Types in Java

There are 8 primitive datatypes in Java:

- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `char`
- `boolean`

Types differ in size (=memory usage) and range.

Primitive Types Table

Table 1: Java primitive types

type	size [B]	range	use
`byte`	1	-128 to 127	very small integer number
`short`	2	-32768 to 32767	small integer number
`int`	4	-2147483648 to 2147483647	integer number
`long`	8	-9223372036854775808 to 9223372036854775807	large integer number
`float`	4	(1.4E-45 to 3.4028235E38)	floating point number
`double`	8	4.9E-324 to 1.7976931348623157E308	higher precision floating point number
`char`	2	0 to 65535	single character
`boolean`	depends	true/false	boolean (logical) value

For further details visit The Java™ Tutorials on data types.

Binary Numbers

- only has two symbols: 0 and 1
- each digits is referred to as a bit

Table 2: binary numbers

binary	decimal
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1111	15

Hexadecimal Numbers

- has 16 symbols: digits from 0 to 9 plus letters A to F
- one hexadecimal symbol can be represented with 4 bits

Table 3: hexadecimal numbers

hexadecimal	decimal	binary
01	1	0000 0001
09	9	0000 1001
0A	10	0000 1010
0B	11	0000 1011
0F	15	0000 1111
10	16	0001 0000
11	17	0001 0001
1A	26	0001 1010
1F	31	0001 1111
FF	255	1111 1111

Two's Complement

- is used to represent signed integers
- as in the binary system each bit is a power of two
- only the most significant bit is the negative power of two
- see example table for three bit signed integers

Table 4: three bit signed integer

decimal	binary	decimal conversion
0	000	$-0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
1	001	$-0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
2	010	$-0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
3	011	$-0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
-4	100	$-1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
-3	101	$-1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
-2	110	$-1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
-1	111	$-1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

Strings

- Strings are not primitive types
- represent a “String of characters”
- sequence of 0 or more characters
- delimited by double quotes (`'"'`)
- empty String `""`
- `"a string of characters"`
- `"42"`

MiniExercise

Which type would you chose for

- a name
- an amount of money
- a count (e.g. the number of visitors)
- an ID
- a color
- a calendar date
- a time of the day
- a day of the week (one of MON-SUN)
- an estimate of the number of protons in the universe
- the mass of the earth

Variables

Variables I

- are objects in memory
- have
  - type
  - value
  - name
  - scope
  - physical location in memory
- must be declared before use
- must be initialized before first use of their value
- can appear on the left side of an assignment (L-value)
- variable name stands for current value of the variable (except when used as L-value)
- can change value through assignment
- variable names must be valid identifiers

Variable declaration

```
TYPE NAME;
```

```
int a; //declared, but not initialized
int b = 4; //declared and initialized
int c, d, e;
int f = 4, g, h = b, i = b + 4;

//a, c, d, e, g are still not initialized
```

(Re-) assignment

```
int a =42;
//...
a = 23; //a new value is stored in a
```

Scope of a variable

- The scope of a variable describes the sections of code in which the variable is *known*.
- A variable is *in scope* in the block within which it is declared.

```
{
    int a = 23;
        { //open a new scope
            int b = 42;
            //a and b are in scope
        }
    //b is no longer in scope
}
```

Identifiers

Overview

- Identifiers are “names” we chose for things like variables, functions and types.
- An identifier can consist of
  - letters [A-Za-z]
  - digits [0-9]
  - underscores [_]
- An identifier cannot start with a digit.
- identifiers are case sensitive

Reserved keywords

The following Java reserved keywords cannot be used as identifiers:

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Java Naming convention

Overview I

- use CamelCase
- choose short, descriptive names
- variable names
  - start with a lower case letter
  - describe the value the variable holds
  - are often nouns
- function names
  - start with a lower case letter
  - are often verbs (or contain verbs)
- class names
  - start in upper case
  - are often nouns
- boolean variables and/or functions
  - often start with “is” or “has”

Overview II

- should make sense (i.e. almost sound English) after an “if” or “while” (e.g. “if isEmpy” or “if hasNext”)

- constants are named in all upper case separated by underscores
  - `DAYS_PER_WEEK`
  - `SECONDS_PER_MINUTE`

Naming tips

Table 5: Variable Naming

name	comment
`n`	good for integers, when meaning is clear from context
`i`	often used as loop counter
`age`	good, clear and short. probably numeric value
`name`	good, clear and short. probably a String
`state`	state of a system. good, clear and short. probably an `int`
`dateOfBirth`	good, clear and short and camelCase
`isValid`	good, clear and short. probably a boolean.
`variable`	we know it's a variable. Describe the value!
`johnny`	bad. meaning unclear.
`value`	bad. meaning unclear.
`test003_4_secondTry_DoesNotWork`	getting frustrated?
`theFirstLetterOfTheLastName`	probably too long.

Expressions

Expressions - Overview I

- An expression describes the calculation of a **value**.
- An expression (`EXPR`) can be
  - a literal
  - a variable
  - `EXPR BINARY_OPERATOR EXPR`
  - `UNARY_OPERATOR EXPR`
  - a function call

Literals

Literals - Overview

`int` literals (examples) I

decimal

- `0`
- `-42`
- `255`

hexadecimal

- `0x0`
- `0x2A` or `0x2A`
- `0xff` or `0xFF`

oktal

- `0`
- `052`
- `0377`

`long` literals (examples) I

- `0L`
- `42L`
- `999_999_999L`

`double` literals (examples) I

- `3.14159`
- `42.0`
- `42.`
- `42D`
- `0.42E2`
- `0.42e2`

`float` literals (examples) I

- `42.0F` or `42.0f`
- `42.F` or `42.f`
- `42F` or `42F`
- `0.42E2f` or- `0.42e2F`

`char` literals (examples) I

- `'H'`
- `'\n'`
- `'\u03C0'` (unicode escape sequence)

`boolean` literals (examples) I

`boolean`:

- `true`
- `false`

Operators and Expressions

Operators - Overview I

- An operator computes a value from one or more values, called *operands*.
- Properties of Operators
  - Arity
  - Precedence
  - Associativity
- Operators in Java
  - Arithmetic
  - Relational
  - Conditional
  - Assignment
  - Bitwise operators
  - Ternary

Arity

- The number of operands an operator takes

Table 6: Arity of operators

number of operands	Arity
1	unary
2	binary
3	ternary
n	n-ary

Precedence

- Determines the order of evaluation of expressions with several operators
- Operators with higher precedence are evaluated before operators with lower precedence

Precedence Table

Table 7: Java Operator Precedence (highest at the top) and Associativity

Precedence Group	Operators	Associativity
postfix	`expr++ expr--`	left
unary	`++expr --expr +expr -expr ~ !`	right
multiplicative	`* / %`	left
additive	`+ -`	left
shift	`<< >> >>>`	left
relational	`< > <= >= instanceof`	left
equality	`== !=`	left
bitwise AND	`&`	left
bitwise exclusive OR	`^`	left
bitwise inclusive OR	`|`	left
logical AND	`&&`	left
logical OR	`||`	left
ternary	`? :`	left
assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`	right

Associativity

- Determines how operators of the same precedence are grouped
- possible associativity values for binary operators are
  - left-associative
  - right-associative

```
//left-associative OPERATORS
EXPR1 OPERATOR_1 EXPR2 OPERATOR_2 EXPR3
//same as ((EXPR1 OPERATOR_1 EXPR2) OPERATOR_2 EXPR3)
```

```
//right-associative OPERATORS
EXPR1 OPERATOR_1 EXPR2 OPERATOR_2 EXPR3
//same as (EXPR1 OPERATOR_1 (EXPR2 OPERATOR_2 EXPR3))
```

Arithmetic Operators

- Arithmetic Operators operate on two (numeric) expressions and return a numeric value.
- binary
- left-associative (`k - i - j;` same as `(k - i) - j;`)

Table 8: Arithmetic Operators

Symbol	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo

Mixed Arithmetic Expressions Rules

- The first rule in the sequence that applies, is carried out.
  - If either operand is of type `double`, the other is converted to `double` before the operation is carried out
  - If either operand is of type `float` the other is converted to `float` before the operation is carried out
  - If either operand is of type `long`, the other is converted to `long` before the operation is carried out
- If neither operand is `double`, `float`, or `long`, they must be `int`, `short`, or `byte`.
- Result of arithmetic expression is at least of type `int`

Simple Assignment Operator I

- Assignment (=) operator stores a value in a variable
- Usage

```
L_VALUE = EXPR;
```

- The left hand side (`L_VALUE`) must be an l-value (a variable)
- The right hand side (`EXPR`) is evaluated first and the result copied to the left hand side
- Types of `L_VALUE` and `EXPR` must be compatible
- Right-associative
- returns value of `EXPR`

```
i = 42; //simple assignment
i = k * 42;

i = j = k = 0; //same as i = (j = (k = 0));
```

Type Cast Operator I

- explicitely converts the type of an expression
- beware: data might be lost
- usage

```
(TYPE_NAME) EXPR
```

- example

```
(int) 3.14159 //"cast" a double value to int
```

Compound Assignment Operators I

- Update a variable
- Usage

```
L_VALUE op= EXPR
// same as L_VALUE = (type of L_VALUE) (LVALUE op EXPR)
```

- beware: casts to type of `L_VALUE`
- Operators: `*=`, `/=`, `+=`, `-=`, `%=`, `&=`, `^=`, `|=`, `<<=`, `>>=`, `>>>=`
- example

```
int i;
//...
i += 2; //same as i = (int) (i + 2);
```

Increment/Decrement Operators

- in/decrease an l-value by one
- unary
- two flavors
  - prefix form
  - postfix form
- usage

```
L_VALUE++  //postfix
++L_VALUE  //prefix
L_VALUE--
--L_VALUE
```

- prefix form returns the *new* vale
- postfix form returns the *old* value (and then in/decrements)

Increment example

```
int i = 1;
int j = 2;
int k = ++i + j++; // i==2 j==3 k==4
i = 1;
j = 2;
k = i++ + j++; // i==2 j==3 k==3
```